

UNIDADE 1



Pilares da OO e UML

Prof. Mauricio Pasetto de Freitas, MSc.
mauriciopasetto@univali.br

Plano de Aula

Objetivo

- ❑ Compreender conceitos básicos da OO e UML.

Conteúdo

- ❑ Pilares da OO.
- ❑ UML
- ❑ Diagrama de Caso de Uso
- ❑ Especificação de Casos de Uso.

Metodologia

- ❑ Aula expositiva;
- ❑ Exercícios.

Avaliação

- ❑ Atividade em Sala.

Bibliografia

- ❑ OBERLEITNER, Allen; MASIERO, Andrey Araujo. Programação orientada a objetos. São Paulo: <i>1 e-book (137 p. (Série Universitária). ISBN 9786555364781.
- ❑ DEITEL, Paul J.; DEITEL, Harvey M. Java TM: como programar São Paulo, SP: Pearson Prentice Hall, 2017.
- ❑ GUEDES, Gilleanes T. A. *UML 2: uma abordagem prática*. 3. ed. São Paulo: Novatec Editora, 2018.

Sumário

- 1** **Pilares da OO**
- 2** **UML – Unified Modeling Language**

A solid blue background featuring a variety of white geometric shapes and symbols. The elements include: a large triangle in the upper left; two right-pointing arrows at the top center; a large six-pointed star in the upper middle; a circle in the upper right; a cluster of four small circles in the middle left; a four-pointed star in the middle right; a cross in the lower middle; a circle in the lower right; a plus sign in the bottom center; a right-pointing arrow in the bottom right; and several other smaller stars and circles scattered throughout.

Definição de OO

“A Orientação a Objetos (OO) é uma técnica de programação baseada na **construção e utilização de objetos**. Um objeto combina **dados e operações específicas**, o que define um conjunto particular de responsabilidades. Um sistema OO é um **conjunto de objetos que se relacionam para produzir os resultados desejados.**”

Junior, Peter Jandl. Java Guia do Programador - 4ª Edição: Atualizado para Java
16. Novatec Editora.

- **A execução do programa** se dá pela interação entre os objetos, mais formalmente pela troca de mensagens entre eles.
- **Envio de mensagem:** acessar um atributo para leitura ou escrita, ou invocar um método de outro objeto.

Pilares da OO:



- Os **pilares da Orientação a Objetos (OO)** são os quatro princípios fundamentais que estruturam o paradigma orientado a objetos.
- Eles definem como os sistemas devem ser modelados, organizados e implementados.

Abstração

“Abstração é o processo de identificar as **características essenciais de um objeto** que o distinguem de todos os outros tipos de objetos e, assim, estabelecer limites conceituais claramente definidos a partir da perspectiva do observador.”

No âmbito da Programação Orientada a Objetos, abstração significa:

- Focar **no que o objeto faz**, e não em como ele faz;
- Destacar apenas os **atributos e comportamentos relevantes** ao problema;
- **Omitir detalhes de implementação** desnecessários naquele nível de análise.

BOOCH, Grady. *Object-Oriented Analysis and Design with Applications*. 3rd ed. Boston: Addison-Wesley, 2007.

Abstração



“Uma classe define a **implementação de um tipo de objeto**, especificando sua representação (dados) e as operações que podem manipulá-lo.”

STROUSTRUP, Bjarne. *The C++ Programming Language*. 4th ed. Boston: Addison-Wesley, 2013.

Abstração

```
public class Animal {

    private int id;
    private String nome;
    private Especie especie;
    private String raca;
    private double peso;
    private Sexo sexo;
    private LocalDate dataNascimento;

    public Animal(int id, String nome, Especie especie, String raca,
        double peso, Sexo sexo, LocalDate dataNascimento) {
        super();
        this.id = id;
        this.nome = nome;
        this.especie = especie;
        this.raca = raca;
        this.peso = peso;
        this.sexo = sexo;
        this.dataNascimento = dataNascimento;
    }

    //Getters e Setters
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getNome() {
        return nome;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
    public Especie getEspecie() {
        return especie;
    }
    public void setEspecie(Especie especie) {
        this.especie = especie;
    }
}
```

```
public Sexo getSexo() {
    return sexo;
}

public void setSexo(Sexo sexo) {
    this.sexo = sexo;
}

public LocalDate getDataNascimento() {
    return dataNascimento;
}

public void setDataNascimento(LocalDate dataNascimento) {
    this.dataNascimento = dataNascimento;
}

@Override
public String toString() {
    return "Animal [id=" + id + ", nome=" + nome + ", especie=" + especie
        + ", raca=" + raca + ", peso=" + peso
        + ", sexo=" + sexo + ", dataNascimento=" + dataNascimento + "]";
}

@Override
public int hashCode() {
    return Objects.hash(id);
}

@Override
public boolean equals(Object obj) {
    if (this == obj)
        return true;
    if (obj == null)
        return false;
    if (getClass() != obj.getClass())
        return false;
    Animal other = (Animal) obj;
    return id == other.id;
}
```

Abstração

```
1 package model;
2
3 public class Cliente {
4     private int id;
5     private String nome;
6     private String email;
7     private String telefone;
8
9     public Cliente(int id, String nome, String email, String telefone) {
10         this.id = id;
11         this.nome = nome;
12         this.email = email;
13         this.telefone = telefone;
14     }
15
16     public int getId() {
17         return id;
18     }
19
20     public void setId(int id) {
21         this.id = id;
22     }
23 }
```

pacote → `package model;`

definição da classe → `public class Cliente {`

atributos → `private int id;`
`private String nome;`
`private String email;`
`private String telefone;`

construtor → `public Cliente(int id, String nome, String email, String telefone) {`

Corpo da classe → `this.id = id;`
`this.nome = nome;`
`this.email = email;`
`this.telefone = telefone;`

métodos → `public int getId() {`
`return id;`
`public void setId(int id) {`
`this.id = id;`

Herança

“Herança é um relacionamento entre classes no qual uma classe **compartilha a estrutura ou o comportamento definido** em uma ou mais outras classes.”

No paradigma orientado a objetos, herança:

- Permite que **uma classe derivada reutilize atributos e métodos** de uma classe base;
- Estabelece uma relação do tipo “**é um**” (is-a);
- Favorece reutilização e **extensão de comportamento**;
- Possibilita **especialização**.

BOOCH, Grady; RUMBAUGH, James; JACOBSON, Ivar. *The Unified Modeling Language User Guide*. 2nd ed. Boston: Addison-Wesley, 2005.

Classe Object:

Module java.base

Package java.util

Class ArrayList<E>

java.lang.Object

java.util.AbstractCollection<E>

java.util.AbstractList<E>

java.util.ArrayList<E>

Type Parameters:

E - the type of elements in this list

Module java.base

Package java.lang

Class Object

java.lang.Object

public class **Object**

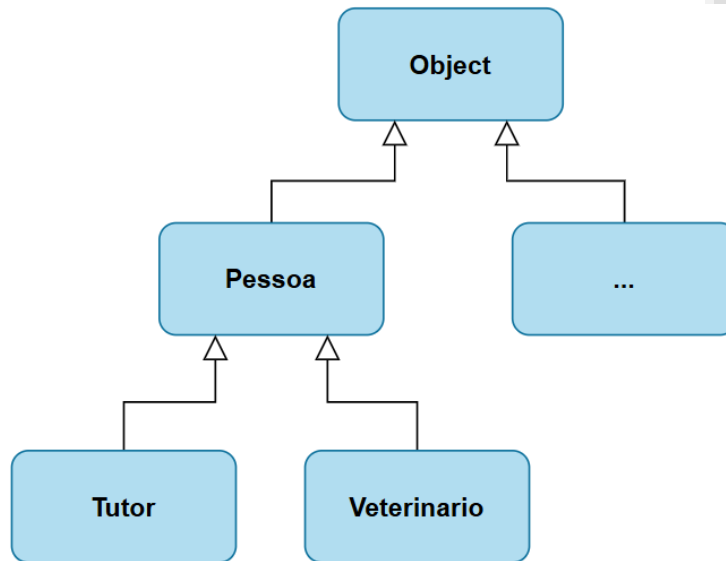
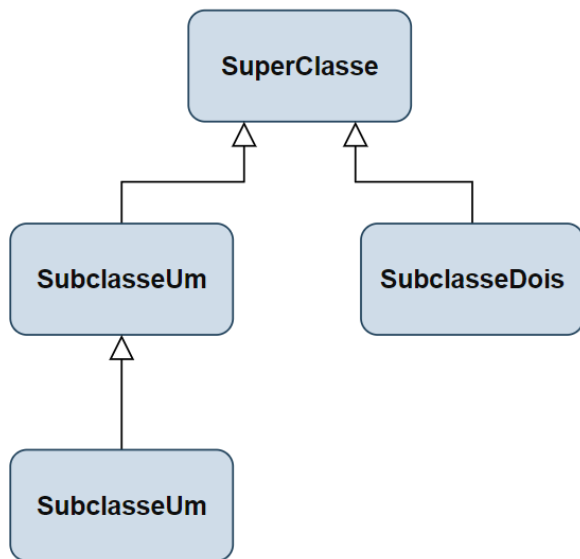
Class Object is the root of the class hierarchy. Every class has Object as a superclass. All objects, including arrays, implement the methods of this class.

Since:

1.0

Herança:

Herança: Herança permite que uma classe (classe filha ou subclasse) herde atributos e métodos de outra classe (classe pai ou superclasse).



Herança: Super Classe Object

Class Object

java.lang.Object

```
public class Object
```

A classe Object é a raiz da hierarquia de classes. Toda classe tem Objectuma superclasse. Todos os objetos, incluindo arrays, implementam os métodos dessa classe.

Desde:

JDK 1.0

Constructors14

Constructor	Description
<code>Object()</code>	

Method Summary

All Methods	Instance Methods	Concrete Methods
Modifier and Type	Method	
protected <code>Object</code>	<code>clone()</code>	
<code>boolean</code>	<code>equals(Object obj)</code>	
protected <code>void</code>	<code>finalize()</code>	
<code>Class<?></code>	<code>getClass()</code>	
<code>int</code>	<code>hashCode()</code>	
<code>void</code>	<code>notify()</code>	
<code>void</code>	<code>notifyAll()</code>	
<code>String</code>	<code>toString()</code>	
<code>void</code>	<code>wait()</code>	
<code>void</code>	<code>wait(long timeout)</code>	
<code>void</code>	<code>wait(long timeout, int nanos)</code>	

```
public abstract class Pessoa extends Object{
```

```
    private int id;
    private String nome;
    private LocalDate dataNascimento;
    private String endereco;
    private String email;
```

```
    public Pessoa(int id, String nome, LocalDate dataNascimento,
        String endereco, String email) {
        super();
        this.id = id;
        this.nome = nome;
        this.dataNascimento = dataNascimento;
        this.endereco = endereco;
        this.email = email;
    }
```

```
@Override
```

```
public String toString() {
    return "Pessoa [id=" + id + ", nome=" + nome +
        ", dataNascimento=" + dataNascimento +
        ", endereco=" + endereco +
        ", email=" + email + "];"
}
```

```
@Override
```

```
public int hashCode() {
    return Objects.hash(email, id);
}
```

```
@Override
```

```
public boolean equals(Object obj) {
    if (this == obj)
        return true;
    if (obj == null)
        return false;
    if (getClass() != obj.getClass())
        return false;
    Pessoa other = (Pessoa) obj;
    return Objects.equals(email, other.email) &&
        id == other.id;
}
```

```
    public int calcularIdade() {
        int idade = Period.between(dataNascimento,
            LocalDate.now()).getYears();
        return idade;
    }
```

```
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getNome() {
        return nome;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
    public LocalDate getDataNascimento() {
        return dataNascimento;
    }
    public void setDataNascimento(LocalDate dataNascimento) {
        this.dataNascimento = dataNascimento;
    }
    public String getEndereco() {
        return endereco;
    }
    public void setEndereco(String endereco) {
        this.endereco = endereco;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
}
```

Herança: Super Classe Pessoa

```

public class Tutor extends Pessoa{

    private String profissao;
    private int diaPagamento;
    private List<Animal> animais;

    public Tutor (int id, String nome, LocalDate dataNascimento,
        String endereco, String email, String profissao, int diaPagamento) {
        super(id, nome, dataNascimento, endereco, email);
        this.profissao = profissao;
        this.diaPagamento = diaPagamento;
        this.animais = new ArrayList<>();
    }

    public String getProfissao() {
        return profissao;
    }
    public void setProfissao(String profissao) {
        this.profissao = profissao;
    }
    public int getDiaPagamento() {
        return diaPagamento;
    }
    public void setDiaPagamento(int diaPagamento) {
        this.diaPagamento = diaPagamento;
    }

    public List<Animal> getAnimais() {
        return animais;
    }

    public void setAnimais(List<Animal> animais) {
        this.animais = animais;
    }

    @Override
    public String toString() {
        return "Tutor [" + super.toString() + ", profissao=" + profissao + ", diaPagamento=" + diaPagamento
            + ", [Animais: " + animais.toString() + " ]";
    }
}

```

Herança: Subclasse Tutor


```

public class Veterinario extends Pessoa{

    public String crmv;
    public Especialidade especialidade;

    public Veterinario(int id, String nome, LocalDate dataNascimento,
        String endereco, String email, String crmv,
        Especialidade especialidade) {
        super(id, nome, dataNascimento, endereco, email);
        this.crmv = crmv;
        this.especialidade = especialidade;
    }

    public String getCrmv() {
        return crmv;
    }
    public void setCrmv(String crmv) {
        this.crmv = crmv;
    }
    public Especialidade getEspecialidade() {
        return especialidade;
    }
    public void setEspecialidade(Especialidade especialidade) {
        this.especialidade = especialidade;
    }

    @Override
    public String toString() {
        return "Veterinario [" + super.toString() + ", crmv=" +
            crmv + ", especialidade=" + especialidade + "]";
    }

    @Override
    public int hashCode() {
        final int prime = 31;
        int result = super.hashCode();
        result = prime * result + Objects.hash(crmv);
        return result;
    }
}

```

```

@Override
public boolean equals(Object obj) {
    if (this == obj)
        return true;
    if (!super.equals(obj))
        return false;
    if (getClass() != obj.getClass())
        return false;
    Veterinario other = (Veterinario) obj;
    return Objects.equals(crmv, other.crmv);
}

```

Herança: Subclasse Veterinario

Herança

Sobreposição de métodos:

Ocorre quando **uma classe filha (subclasse)** fornece uma **nova implementação para um método já definido na classe pai (superclasse)**, mantendo **a mesma assinatura do método**.

Para que haja sobreposição em Java:

- ❑ O método deve ter **mesmo nome**
- ❑ Deve ter **mesma lista de parâmetros (assinatura)**
- ❑ Deve ter **mesmo tipo de retorno**
- ❑ Deve estar em uma **relação de herança**
- ❑ Recomendado o uso da anotação **@Override**



Definição na superclasse Object:

```
public String toString() {  
    return getClass().getName() + "@" + Integer.toHexString(hashCode());  
}
```

```
public boolean equals(Object obj) {  
    return (this == obj);  
}
```

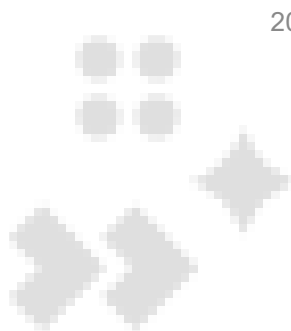
```
public native int hashCode();
```

Classe Pessoa sobreposição de métodos:

```
@Override
public String toString() {
    return "Pessoa [id=" + id + ", nome=" + nome +
        ", dataNascimento=" + dataNascimento +
        ", endereco=" + endereco +
        ", email=" + email + "];"
}
```

```
@Override
public int hashCode() {
    return Objects.hash(email, id);
}
```

```
@Override
public boolean equals(Object obj) {
    if (this == obj)
        return true;
    if (obj == null)
        return false;
    if (getClass() != obj.getClass())
        return false;
    Pessoa other = (Pessoa) obj;
    return Objects.equals(email, other.email) &&
        id == other.id;
}
```



Herança

Sobrecarga de métodos (não depende de herança):

Ocorre quando existem **múltiplos métodos com o mesmo nome na mesma classe**, mas com **assinaturas diferentes (parâmetros diferentes)**.

Características

- ❑ Mesmo nome de método
- ❑ Diferente **lista de parâmetros**:
 - quantidade
 - tipo
 - ordem
- ❑ Ocorre na **mesma classe** (não depende de herança)
- ❑ A escolha do método é feita em **tempo de compilação (compile-time)**

Herança

Sobrecarga de métodos:

```
class Calculadora {  
  
    int somar(int a, int b) {  
        return a + b;  
    }  
  
    double somar(double a, double b) {  
        return a + b;  
    }  
  
    int somar(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

Encapsulamento

“Encapsulamento é o processo de **compartimentalizar os elementos de uma abstração** que constituem sua estrutura e comportamento; **ele separa a interface contratual de uma abstração de sua implementação.**”

Encapsulamento implica em:

- Separar o que o objeto oferece (interface pública)
- De como ele realiza internamente suas operações (implementação)
- Proteger o estado interno contra acesso indevido
- Estabelecer um “contrato” claro entre objeto e cliente

BOOCH, Grady. *Object-Oriented Analysis and Design with Applications*.
3rd ed. Boston: Addison-Wesley, 2007.

Encapsulamento

Modificadores de acesso:

- A visibilidade é utilizada para indicar o nível de acessibilidade de um determinado atributo ou método, sendo representada à esquerda destes. Existem basicamente quatro modos de visibilidade: **public**, **protected**, **private** e **package (default)**.

private (-): significa que somente os objetos da classe detentora do atributo ou método poderão enxergá-lo.

protected (#): determina que, além dos objetos da classe detentora do atributo ou método, também os objetos de suas subclasses poderão ter acesso a este.

public (+): determina que o atributo ou método pode ser utilizado por qualquer objeto.

Package/default (~): determina que o atributo ou método é visível por qualquer objeto dentro do pacote.

Encapsulamento

1. Modificador de acesso **public**:

Definição:

- ❑ O elemento pode ser acessado **de qualquer lugar**.

Acesso permitido

- ❑ Mesma classe
- ❑ Mesmo pacote
- ❑ Subclasses (mesmo em outros pacotes)
- ❑ Qualquer classe em qualquer pacote

Uso típico

- ❑ APIs públicas
- ❑ Métodos que devem ser amplamente acessíveis



Encapsulamento

1. Modificador de acesso **public**:

Diferentes pacotes, dentro da classe,
em herança **V**:

```
package usuarios;

public class Pessoa {

    public String nome;

    public void exibirNome() {
        System.out.println(nome);
    }
}
```

Dentro da própria classe **V**

```
package pessoal;

import usuarios.Pessoa;

class TestePessoa {

    public static void main(String[] args) {
        Pessoa p = new Pessoa();
        p.nome = "Maria";
        p.exibirNome(); // acesso permitido de qualquer lugar
    }
}
```

Encapsulamento

2. Modificador de acesso **private**:

Definição:

- ❑ O elemento pode ser acessado **apenas dentro da própria classe**.

Acesso permitido

- ❑ Somente na mesma classe

Uso típico

- ❑ Atributos (encapsulamento)
- ❑ Métodos internos auxiliares



Encapsulamento

2. Modificador de acesso **private**:

Mesmo pacote **X**

```
package usuarios;

public class Pessoa {

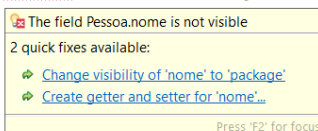
    private String nome;

    public void exibirNome() {
        System.out.println(nome);
    }
}
```

```
package usuarios;

class TestePessoa {

    public static void main(String[] args) {
        Pessoa p = new Pessoa();
        p.nome = "Maria"; // visível apenas dentro da classe
        p. // visível permitido de qualquer lugar
    }
}
```



Dentro da própria classe **V**

Encapsulamento

2. Modificador de acesso **private**:

```
package usuarios;

public class Pessoa {

    private String nome;

    public void exibirNome() {
        System.out.println(nome);
    }
}
```

Dentro da própria classe **V**

Diferente pacote **X**

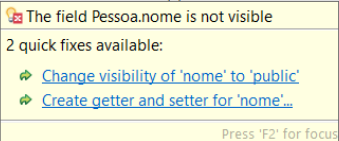
```
package pessoal;

import usuarios.Pessoa;

class TestePessoa {

    public static void main(String[] args) {
        Pessoa p = new Pessoa();
        p.nome = "Maria"; // acesso somente dentro da classe
        p.

        // acesso permitido de qualquer lugar
    }
}
```



Encapsulamento

2. Modificador de acesso **private**:

```
package usuarios;

public class Pessoa {

    private String nome;

    public void exhibirNome() {
        System.out.println(nome);
    }
}
```

Dentro da própria classe **V**

Em herança **X**

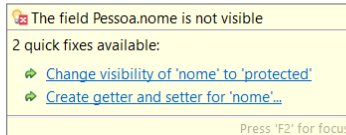
```
package usuarios;

import usuarios.Pessoa;

public class PessoaFisica extends Pessoa{

    private String cpf;

    @Override
    public void exhibirNome() {
        // TODO Auto-generated method stub
        System.out.println("PessoaFisica[ nome:"
            + super.nome+", cpf:"+cpf+"]");
    }
}
```



Encapsulamento

3. Modificador de acesso **protected**:

Definição:

- ❑ O elemento pode ser acessado **no mesmo pacote e também por subclasses**, mesmo que estejam em outros pacotes.

Acesso permitido

- ❑ Mesma classe
- ❑ Mesmo pacote
- ❑ Subclasses (inclusive em outros pacotes)

Uso típico

- ❑ Métodos que devem ser herdados
- ❑ Extensão controlada de comportamento



Encapsulamento

3. Modificador de acesso **protected**:

No mesmo pacote **V**

```
package usuarios;

public class Pessoa {

    protected String nome;

    public void exibirNome() {
        System.out.println(nome);
    }
}
```

```
package usuarios;

class TestePessoa {

    public static void main(String[] args) {
        Pessoa p = new Pessoa();
        p.nome = "Maria"; // acesso somente dentro da classe
        p.exibirNome(); // acesso permitido de qualquer lugar
    }
}
```

Dentro da própria classe **V**

Encapsulamento

3. Modificador de acesso **protected**:

Pacote diferente **X**

```
package usuarios;

public class Pessoa {

    protected String nome;

    public void exibirNome() {
        System.out.println(nome);
    }
}
```

Dentro da própria classe **V**

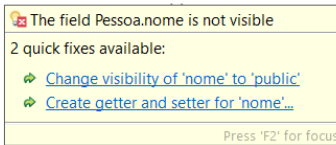
```
package pessoal;

import usuarios.Pessoa;

class TestePessoa {

    public static void main(String[] args) {
        Pessoa p = new Pessoa();
        p.nome = "Maria"; // acesso somente dentro da classe
        p.

    }
}
```



Encapsulamento

3. Modificador de acesso **protected**:

Com uso de herança V

```
package usuarios;

public class Pessoa {

    protected String nome;

    public void exibirNome() {
        System.out.println(nome);
    }
}
```

Dentro da própria classe V

```
package pessoal;

import usuarios.Pessoa;

public class PessoaFisica extends Pessoa{

    private String cpf;

    @Override
    public void exibirNome() {
        // TODO Auto-generated method stub
        System.out.println("PessoaFisica[ nome:"+ super.nome+", cpf:"+cpf+"]");
    }
}
```



Encapsulamento

4. Modificador de acesso **package** (default):

Definição:

- ❑ Quando nenhum modificador é declarado, o acesso é restrito ao **mesmo pacote**.

Acesso permitido

- ❑ Mesma classe
- ❑ Mesmo pacote

Uso típico

- ❑ Classes e métodos internos de um pacote
- ❑ Implementações que não devem ser expostas externamente



Encapsulamento

4. Modificador de acesso package (default):

Mesmo pacote V

```
package usuarios;

public class Pessoa {

    String nome;

    public void exibirNome() {
        System.out.println(nome);
    }
}
```

```
package usuarios;

class TestePessoa {

    public static void main(String[] args) {
        Pessoa p = new Pessoa();
        p.nome = "Maria"; // acesso somente dentro do mesmo pacote
        p.exibirNome(); // acesso permitido de qualquer lugar
    }
}
```

Dentro da própria classe V



Encapsulamento

4. Modificador de acesso package (default):

```
package usuarios;

public class Pessoa {

    String nome;

    public void exibirNome() {
        System.out.println(nome);
    }
}
```

Dentro da própria classe **V**

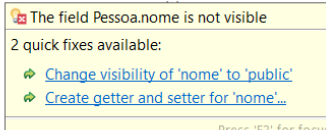
Pacote diferente **X**

```
package pessoal;

import usuarios.Pessoa;

class TestePessoa {

    public static void main(String[] args) {
        Pessoa p = new Pessoa();
        p.nome = "Maria"; // acesso somente dentro do mesmo pacote
        // acesso permitido de qualquer lugar
    }
}
```



Encapsulamento

4. Modificador de acesso package (default):

Uso de herança X

```
package usuarios;

public class Pessoa {

    String nome;

    public void exibirNome() {
        System.out.println(nome);
    }
}
```

Dentro da própria classe V

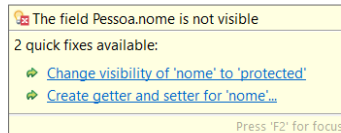
```
package pessoal;

import usuarios.Pessoa;

public class PessoaFisica extends Pessoa{

    private String cpf;

    @Override
    public void exibirNome() {
        // TODO Auto-generated method stub
        System.out.println("PessoaFisica[ nome:"+ super.nome+", cpf:"+cpf+"]");
    }
}
```



Polimorfismo

“Polimorfismo é a existência de diferentes formas; em sistemas orientados a objetos, denota a capacidade de **tratar objetos de diferentes tipos de maneira uniforme.**”

Segundo essa perspectiva:

- Diferentes classes podem responder à **mesma mensagem**;
- O código pode operar sobre um **tipo mais geral** (superclasse ou interface);
- O comportamento específico será determinado **em tempo de execução**.

BOOCH, Grady. *Object-Oriented Analysis and Design with Applications*. 3rd ed. Boston: Addison-Wesley, 2007.

Polimorfismo

```
public class MainPolimorfismo {  
  
    public static void main(String[] args) {  
  
        Pessoa pessoa1 = new Pessoa(1, "Marcelo", LocalDate.of(1977, 11, 16),  
            "Rua Jose Beiro", "mauricio@email.com");  
        Tutor tutor1 = new Tutor(2, "Ana Paula", LocalDate.of(1985, 3, 22),  
            "Rua das Flores", "ana.paula@email.com", "Veterinária", 8);  
        Veterinario veterinario1 = new Veterinario(1, "Alberto", LocalDate.of(1972, 02, 19),  
            "Rua das Araucarias", "alberto@email.com", "123456", Especialidade.OTORRINO);  
  
        List<Object> objetos = new ArrayList();  
        objetos.add(pessoa1);  
        objetos.add(tutor1);  
        objetos.add(veterinario1);  
  
        for (Object objeto : objetos) {  
            System.out.println(objeto.toString());  
        }  
  
        System.out.println(pessoa1.equals(tutor1));  
    }  
}
```

Pessoa [id=1, nome=Marcelo, dataNascimento=1977-11-16, endereco=Rua Jose Beiro, email=mauricio@email.com]

Tutor [Pessoa [id=2, nome=Ana Paula, dataNascimento=1985-03-22, endereco=Rua das Flores, email=ana.paula@email.com], profissao=Veterinária, diaPagamento=8, [Animal]]

Veterinario [Pessoa [id=1, nome=Alberto, dataNascimento=1972-02-19, endereco=Rua das Araucarias, email=alberto@email.com], crmv=123456, especialidade=OTORRINO]

false

Pilares da OO:

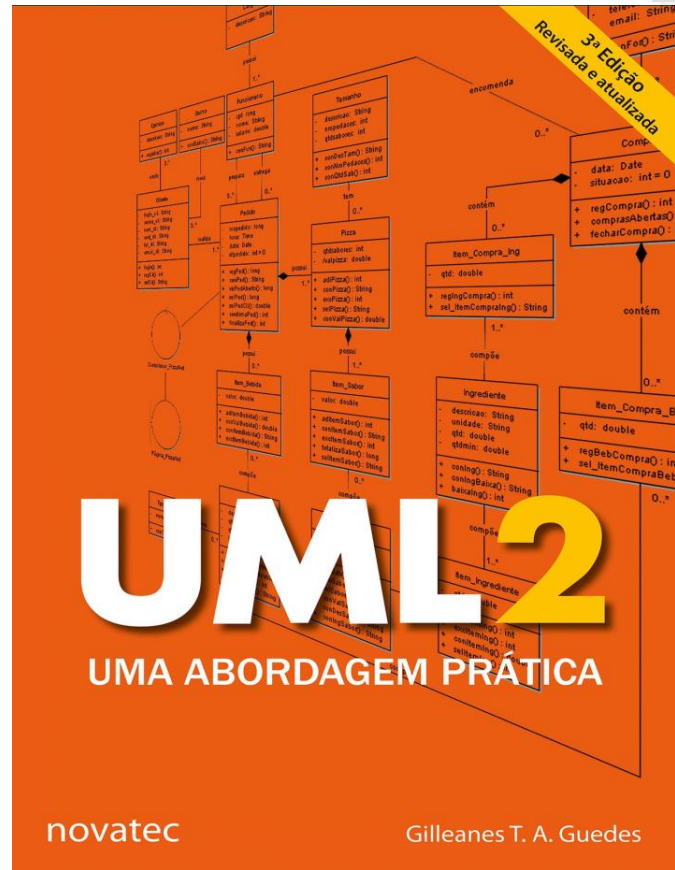


Os pilares da Orientação a Objetos (OO)

A blue background with various white geometric shapes and symbols scattered across it. The symbols include: a large triangle at the top left; two right-pointing chevrons at the top center; a circle at the top right; a large six-pointed star in the upper middle; a cluster of four dots in the upper right; a four-pointed star in the middle right; a cross in the middle right; a small four-pointed star in the lower middle; a circle at the lower middle right; a cluster of four dots in the lower middle; a large six-pointed star at the bottom right; a cross at the bottom center; two right-pointing chevrons at the bottom right; a four-pointed star at the bottom left; and a cluster of four dots at the bottom right.

Referência

GUEDES, Gilleanes T. A. *UML 2: uma abordagem prática*. 3. ed. São Paulo: Novatec Editora, 2018.

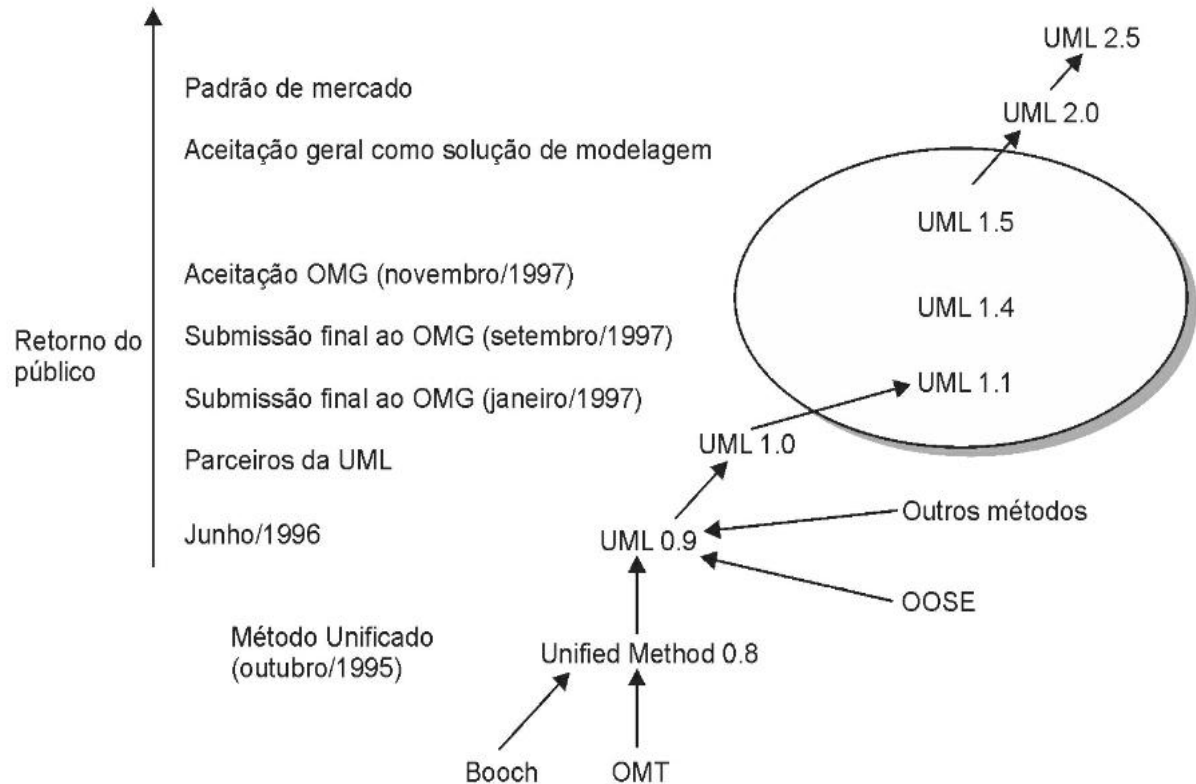


UML – Unified Modeling Language

- É uma **linguagem visual** utilizada para **modelar softwares** baseados no paradigma de **orientação a objetos**;
- É uma linguagem de modelagem de **propósito geral** que pode ser aplicada a todos os domínios de aplicação;
- Essa linguagem é atualmente a **linguagem-padrão de modelagem adotada internacionalmente** pela indústria de engenharia de software.

UML – Unified Modeling Language

Desde 1997, a responsabilidade pela evolução da UML ficou a cargo da **OMG** (Grupo de Gerenciamento de Objeto), órgão aprovador.



UML

[ABOUT US](#) ▾[GROUPS](#) ▾[CERTIFICATIONS](#) ▾[RESOURCES](#) ▾[SPECIFICATIONS](#) ▾[COMMUNITIES](#) ▾[MEMBERSHIP](#) ▾

ABOUT THE UNIFIED MODELING LANGUAGE SPECIFICATION VERSION 2.5.1

[2.5.1](#) • [UML](#) • [SPECIFICATIONS](#)

UML®

Unified Modeling Language

A specification defining a graphical language for visualizing, specifying, constructing, and documenting the artifacts of distributed object systems.



Specification

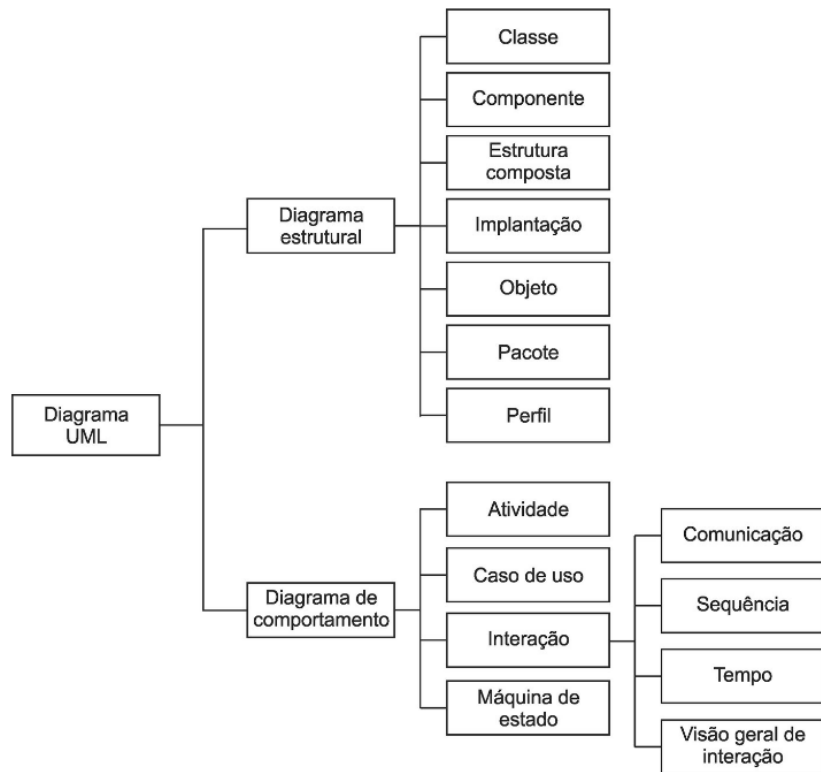
Title:	Unified Modeling Language
Acronym:	UML®
Version:	2.5.1
Document Status:	formal ⓘ
Publication Date:	dezembro 2017
Categories:	Modeling Software Engineering Platform
IPR Mode ⓘ	RF-Limited ⓘ

<https://www.omg.org/spec/UML/>



Escola
Politécnica

UML



Tipos de diagramas:

Diagramas Estruturais: descrevem os elementos estruturais que compõem o sistema;

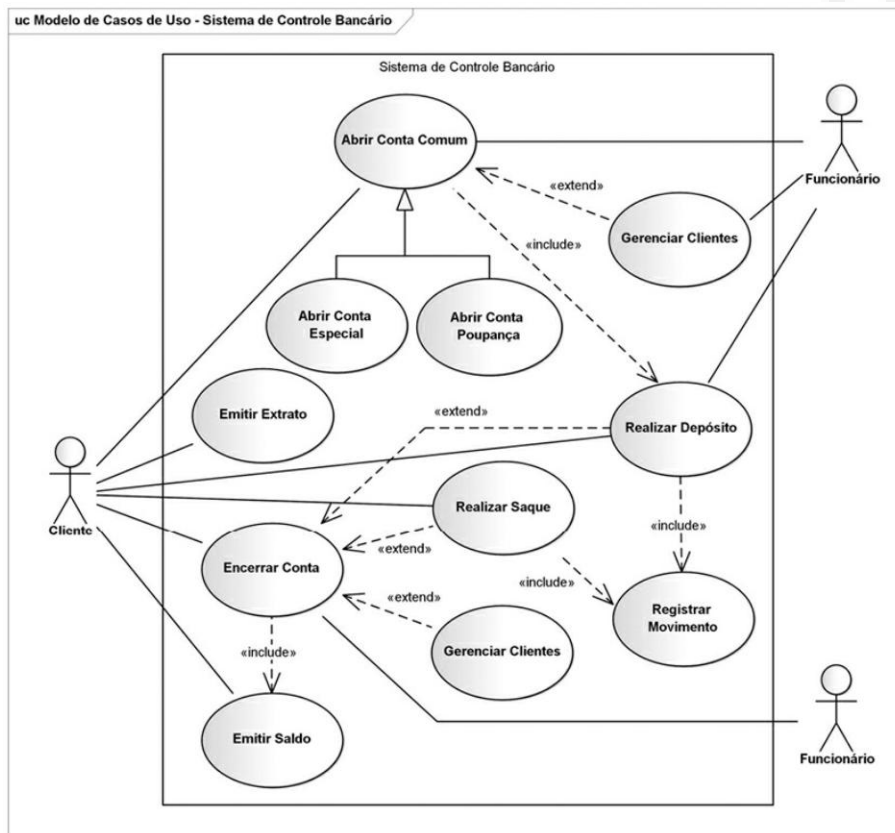
Diagramas de comportamento: descrevem o comportamento dos elementos e suas interações.

Diagramas da UML

UML

Diagrama de Casos de Uso:

- Apresenta uma visão externa geral das funcionalidades que o sistema deverá oferecer aos usuário;

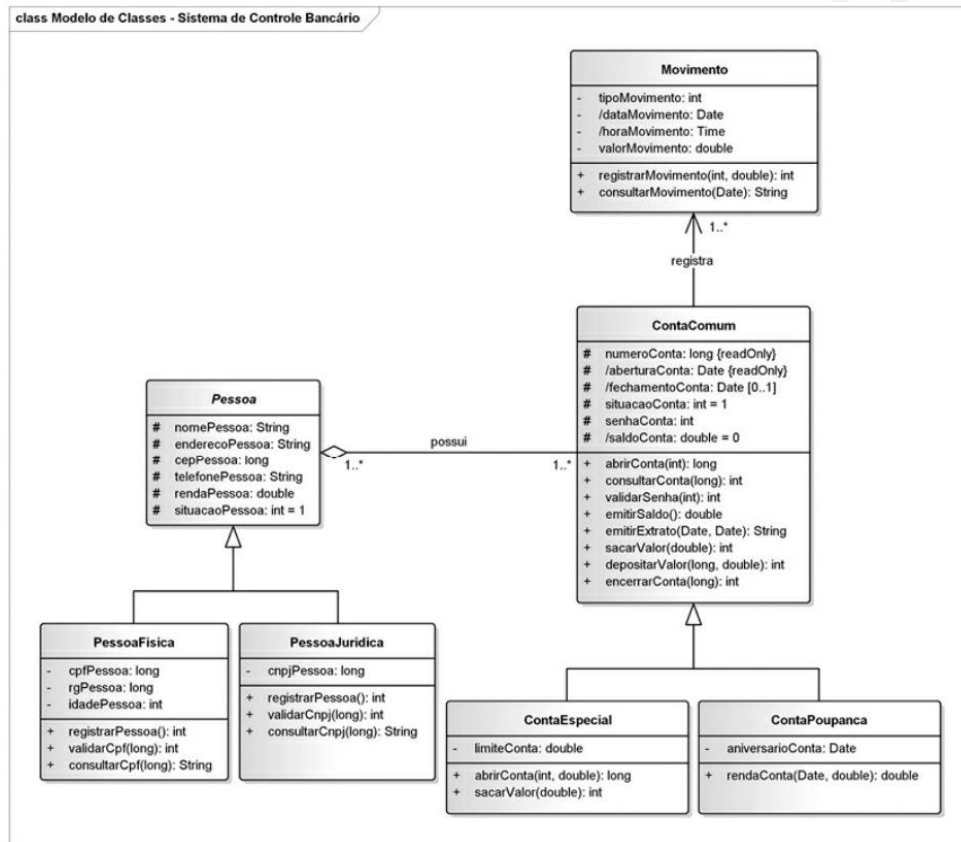


Diagramas de Caso de Uso

UML

Diagrama de Classe:

- Permiti a **visualização das classes que comporão o sistema** com seus respectivos atributos e métodos;
- Demonstrar **como as classes do diagrama se relacionam**, complementam e transmitem informações.

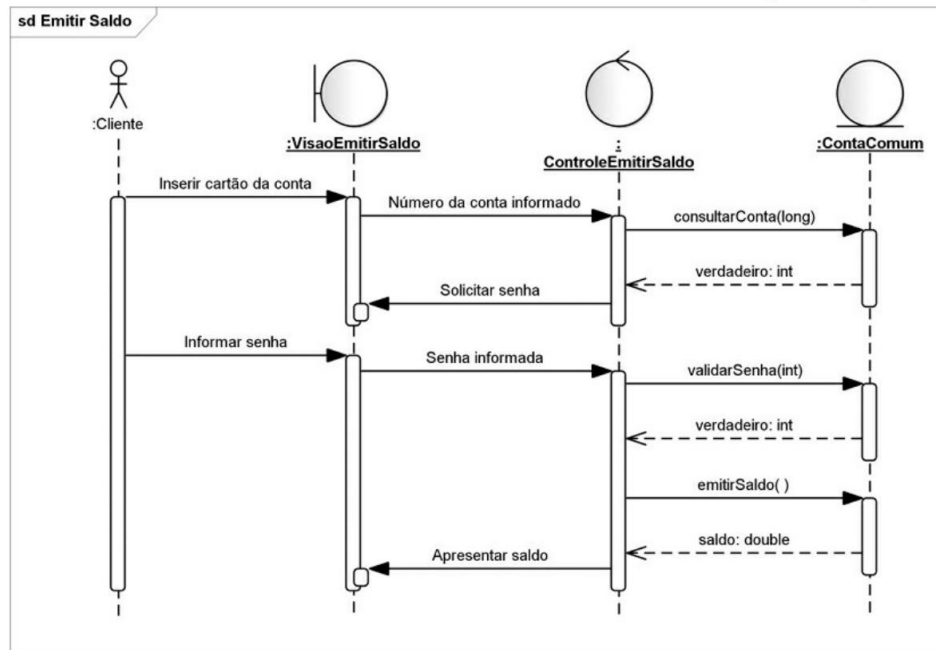


Diagramas de Classe

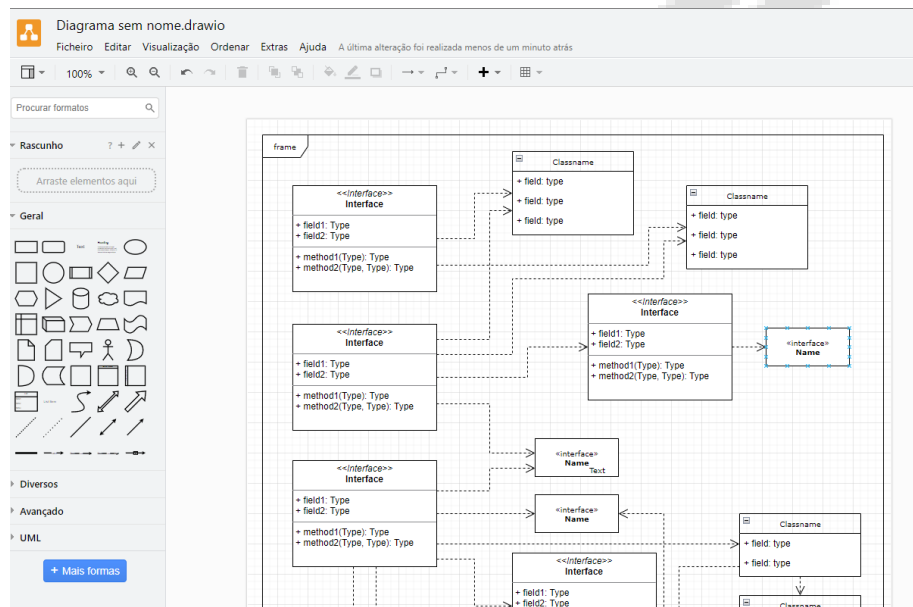
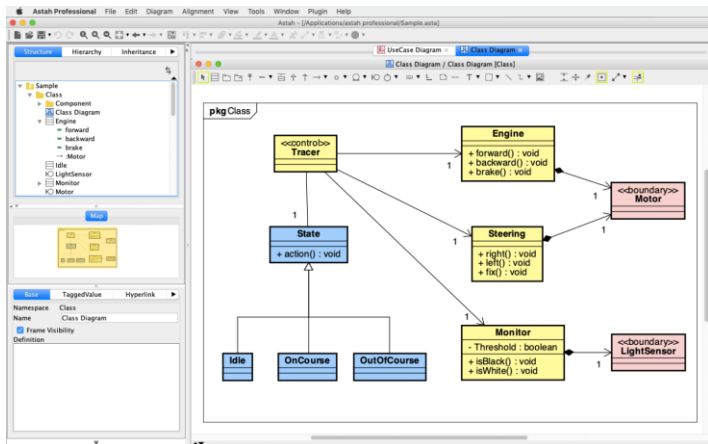
UML

Diagrama de Sequência:

- Procura determinar a **sequência de eventos que ocorrem** em um determinado processo, identificando quais **mensagens devem ser disparadas entre os elementos** envolvidos e em que ordem.
- **Baseia-se no diagrama de casos de uso**, havendo normalmente um diagrama de sequência para cada caso de uso declarado;



Diagramas de Sequência



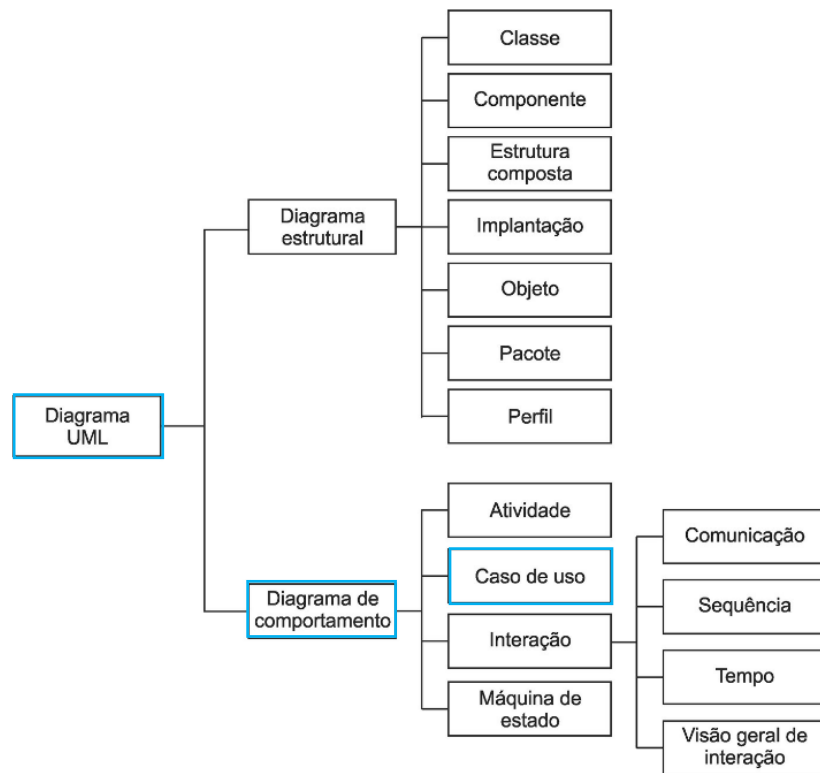
A blue background with various white geometric shapes and symbols scattered across it. The symbols include: a large triangle at the top left; two right-pointing chevrons at the top center; a circle at the top right; a large six-pointed star in the upper middle; a four-dot pattern at the middle left; a four-pointed star at the middle right; a cross at the lower middle; a circle at the bottom left; a plus sign at the bottom center; a right-pointing chevron at the bottom right; a four-dot pattern at the bottom left; a four-pointed star at the bottom center; and a large six-pointed star at the bottom right.

UML – Diagrama de Caso de Uso

“..a visão de caso de uso mostra conceitualmente o **conjunto de funções** que o sistema deve executar para **atender aos requisitos do cliente**, servindo como um **contrato entre o cliente e o desenvolvedor**.”

ADILSON DA SILVA LIMA. UML 2.5 - do Requisito a Solução (Portuguese Edition) (p. 69). Editora Saraiva. Edição do Kindle.

UML – Diagrama de Caso de Uso



UML – Unified Modeling Language

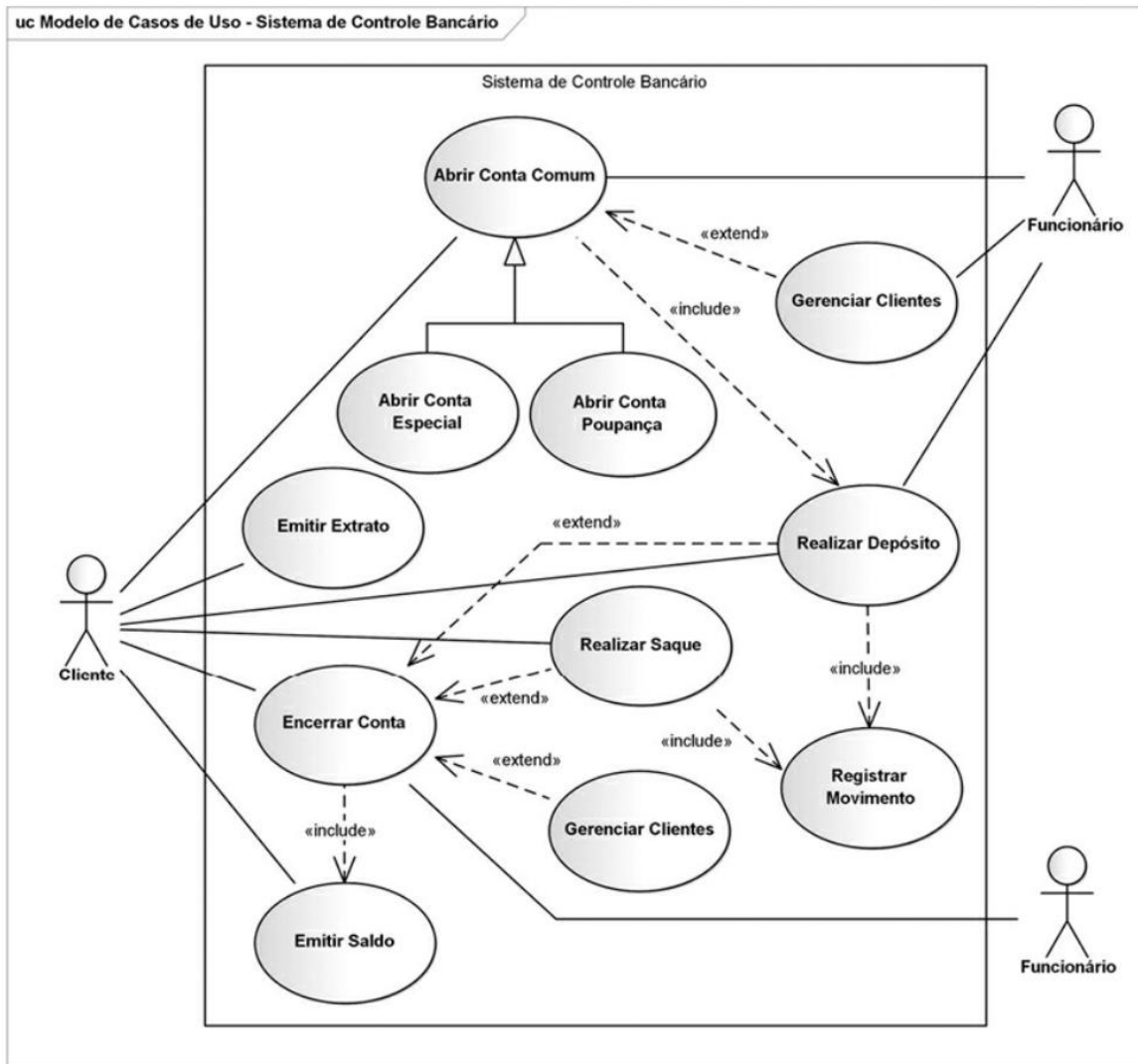
Diagrama de Caso de Usos:
Faz parte do conjunto de diagramas de comportamento propostos pela UML (Unified Modeling Language).

Diagramas da UML

UML – Diagrama de Caso de Uso

Elementos:

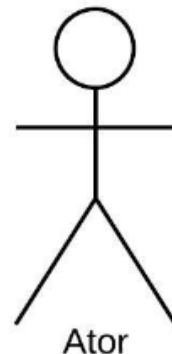
- Ator
- Caso de Uso
- Relacionamentos



UML – Diagrama de Caso de Uso

Ator:

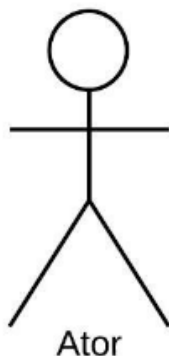
- **Especifica um papel** que é desempenhado por um usuário ou qualquer outro sistema que interage com o sistema que está sendo construído.
 - Pode ser um **hardware** ou outro **software**.
- Reside **fora das fronteiras da aplicação**.
- Todo ator **deve ter um nome** associado a ele.
 - Não é recomendado utilizar a denominação “Usuário”, pois é muito genérica.



UML – Diagrama de Caso de Uso

Ator:

- Um ator pode ser **primário** (quando dá início à ação) ou **secundário** (quando participa da ação ou apenas recebe os resultados da ação).
 - Não existe diferença na sua representação.



UML – Diagrama de Caso de Uso

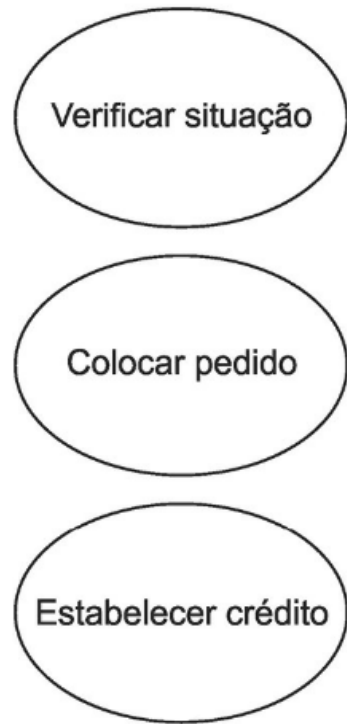
Caso de uso:

- É uma **execução completa de uma funcionalidade** que entrega um valor para o usuário que a utiliza.
- Referem-se aos **serviços, tarefas ou funções** que podem ser utilizados pelos atores.
- São usados para expressar e documentar os **comportamentos das funções do sistema**.
- Os **casos de uso são funcionalidades** que o sistema deve prover, portanto, estamos falando de requisitos funcionais.

UML – Diagrama de Caso de Uso

Caso de uso:

- É representado por **uma elipse com seu nome** dentro ou logo abaixo.
- Representa uma ação e, portanto, deve ser nomeado com um **verbo no infinitivo**.
- Ele faz parte do sistema que está sendo definido e, portanto, **fica dentro do retângulo que representa o sistema**, ao contrário dos atores, que ficam do lado de fora.



UML – Diagrama de Caso de Uso

Caso de uso:

- Quando o sistema tiver **muitas funcionalidades**, pode-se optar por **representar apenas os casos de uso que sejam mais importantes** e que representem a razão de existir do sistema, os chamados casos de uso primários.
- **Apenas o diagrama não é suficiente** para expressar toda a complexidade de uma funcionalidade.
- É necessário que seja elaborada **uma especificação de casos de uso**, que vai detalhar os passos e condições de execução dos casos de uso.

UML – Diagrama de Caso de Uso



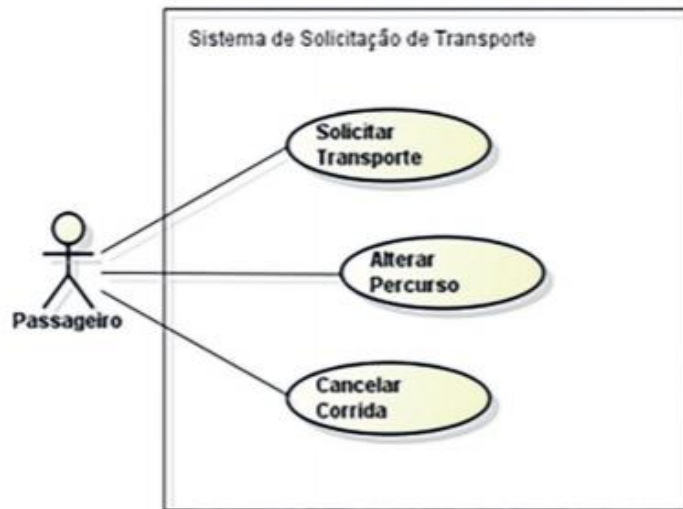
Relacionamentos:

- Entre os **atores e os casos de uso**, entre **casos de uso entre si** e também entre **atores entre si**.
- **Tipos:**
 - Associação;
 - Include;
 - Extend;
 - Generalização.

UML – Diagrama de Caso de Uso

Relacionamento Associação:

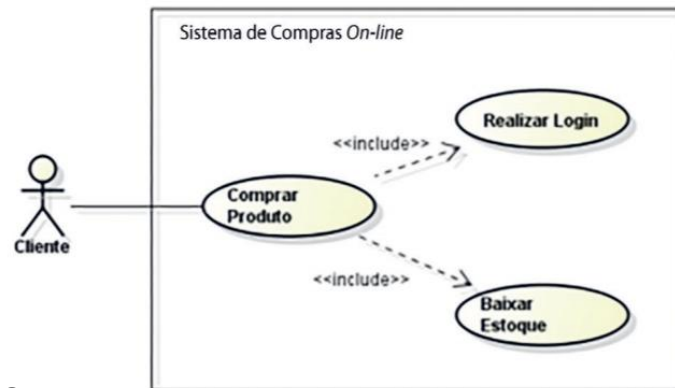
- Indica que o **ator inicia o caso de uso** ou que o ator **recebe os resultados do caso** de uso que foi iniciado por outro ator.
- Representada por **uma linha contínua**.
- É possível que um caso de uso **tenha mais do que um ator** associado.



UML – Diagrama de Caso de Uso

Relacionamento Include:

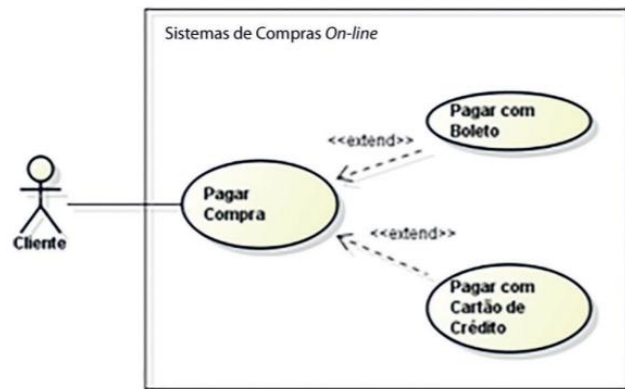
- É utilizado quando o comportamento de um caso de uso **está sendo incluído** dentro de outro caso de uso.
- Ele pressupõe que toda vez que o caso de uso que inclui for executado, **o caso de uso incluído também será**.
 - Será executado de uma vez só.
- Representado por uma seta pontilhada que parte do caso de uso que inclui para o caso de uso que é incluído.
 - Pode-se adicionar o texto <<include>>..



UML – Diagrama de Caso de Uso

Relacionamento Extend:

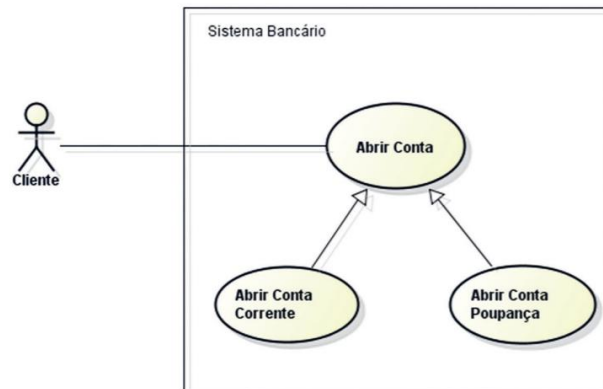
- É utilizado quando o comportamento de um caso de uso estende o comportamento de outro caso de uso.
 - É usado quando o comportamento é **adicionado de forma condicional**.
- ❑ É representado por seta pontilhada que parte do caso de uso que estende para o caso de uso que é estendido.
 - ❑ Pode-se adicionar o texto <<extend>>.



UML – Diagrama de Caso de Uso

Relacionamento Generalização:

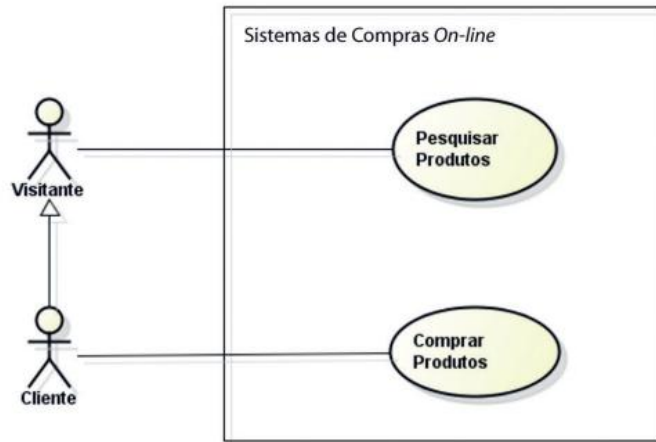
- É utilizado quando **o comportamento** de um caso de **uso é herdado por outro caso de uso** (herança da orientação a objetos).
- Os casos de uso especializados **herdam todos os relacionamentos do caso de uso geral** ao qual estão associados.



UML – Diagrama de Caso de Uso

Relacionamento Generalização:

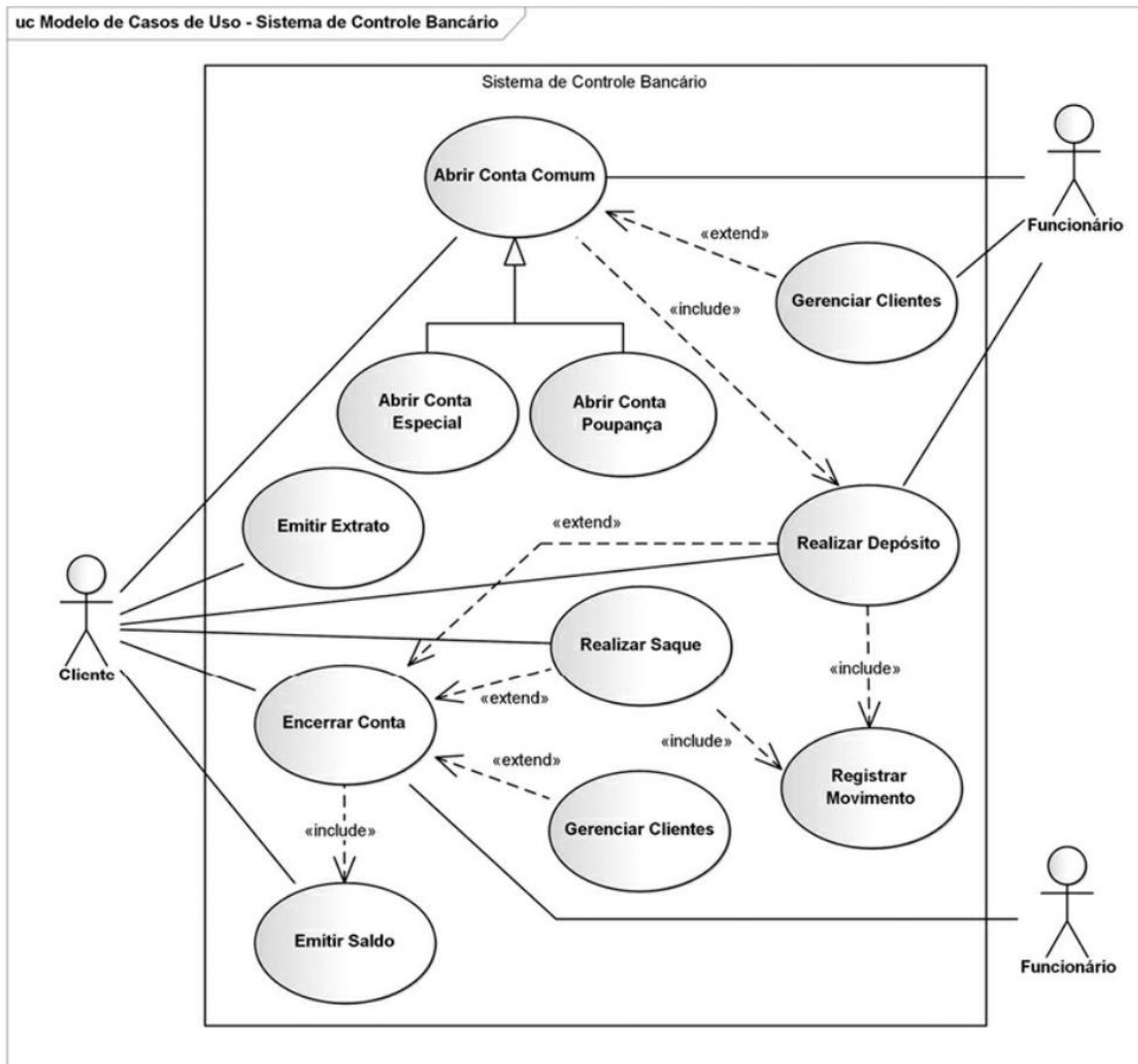
- O relacionamento do tipo generalização **também é aplicável para os atores**, e o conceito é exatamente o mesmo.
- Um ator especializado herda todos os casos de uso e os relacionamentos do ator geral.



UML – Diagrama de Caso de Uso

Elementos:

- Ator
- Caso de Uso
- Relacionamentos



Exercício 01 - Criar um Diagrama de Caso de Uso, considerando os seguintes requisitos:

- O celular oferece o serviço de realizar chamadas, no qual o usuário deve informar um telefone para que o celular ligue. O celular deve registrar as últimas chamadas.
- Semelhante ao serviço de chamadas, o telefone oferece o serviço de mensagens, em que o usuário deve informar o número de telefone para o qual deseja enviar a mensagem. O celular deve igualmente registrar as últimas mensagens.
- O aparelho oferece o serviço de agenda, por meio do qual é possível cadastrar os diversos contatos do usuário. Cada contato armazena o nome do contato e seu telefone. Caso o usuário consulte um telefone já existente, ele poderá ligar para esse contato ou enviar uma mensagem. O sistema deve guardar as últimas ligações feitas, bem como as últimas mensagens enviadas.
- O celular oferece também o serviço de recebimento de chamadas. O sistema deve avisar o recebimento de uma chamada por meio do toque de uma música e o usuário pode aceitar a chamada ou não. As últimas ligações também devem ser gravadas.
- Da mesma forma, o sistema deve oferecer o serviço de recebimento de mensagens, devendo também registrar as últimas mensagens recebidas.
- O celular oferece ainda o serviço de despertador, no qual o usuário pode cadastrar e/ou ativar um ou mais horários para despertar.
- Finalmente, o sistema oferece o serviço de tons, no qual o usuário pode selecionar, entre muitas músicas possíveis, a que mais lhe agrada para avisá-lo do recebimento de uma chamada ou mensagem, ou para despertá-lo.

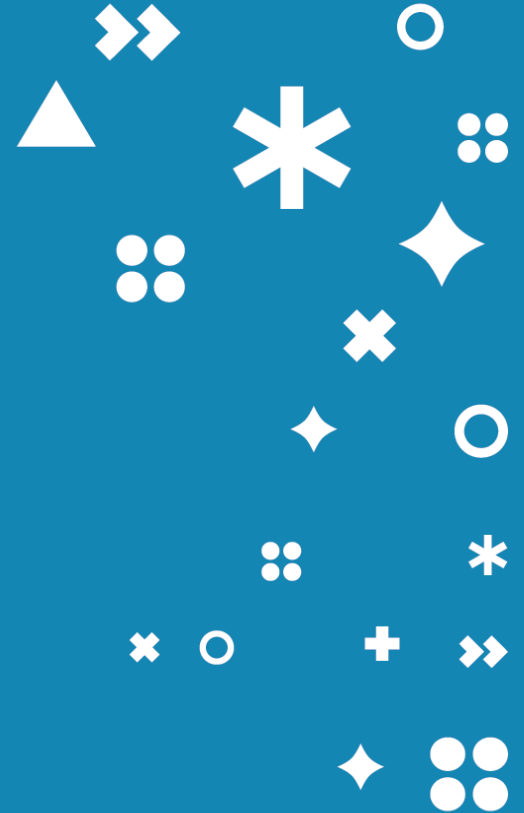
Exercício 02 - Criar um Diagrama de Caso de Uso, considerando os seguintes requisitos:

- O cliente explicou que a clínica veterinária precisa de um sistema que permita o cadastro e gerenciamento de informações sobre os tutores e seus respectivos animais. Cada tutor pode ter mais de um animal cadastrado, e é importante que o sistema permita visualizar facilmente essa relação. Além disso, os veterinários que realizam os atendimentos também devem ter seus dados armazenados, incluindo especialidades e horários de atendimento. Há também a necessidade de cadastrar informações sobre a atendente da clínica, que será responsável por agendar consultas e organizar o fluxo de atendimento.
- Durante as consultas (atendimentos), o sistema deve permitir que o veterinário emita receitas e faça solicitações de exames para os animais atendidos. Foram mencionados dois tipos de receitas: a receita tradicional, que é entregue diretamente ao cliente, e a receita online, que precisa ser integrada com um sistema externo para emissão. O cliente reforçou que essa integração deve ser feita de forma automática e confiável, garantindo que as receitas online sejam geradas corretamente sem necessidade de ações manuais extras por parte dos veterinários.

Exercício 02 - Criar um Diagrama de Caso de Uso, considerando os seguintes requisitos:

- Além disso, todas as consultas realizadas devem gerar um histórico detalhado, contendo informações sobre os procedimentos realizados, receitas emitidas e exames solicitados. O histórico deve estar vinculado ao animal e ser acessível para consultas futuras, permitindo que os veterinários acompanhem a evolução do paciente ao longo do tempo. O cliente destacou que essa funcionalidade é essencial para garantir um atendimento de qualidade e dar suporte a decisões médicas baseadas no histórico do animal.
- Quanto aos requisitos não funcionais, o cliente mencionou que o sistema precisa ser intuitivo e fácil de usar, pois será operado por diferentes usuários com níveis variados de familiaridade com tecnologia. Além disso, destacou a importância da segurança das informações, garantindo que apenas usuários autorizados tenham acesso ao sistema. O sistema também precisa ser acessível via web e por meio de dispositivos móveis, garantindo que os veterinários possam consultar informações tanto na clínica quanto remotamente, se necessário.

Especificação de Casos de Uso



Especificação de Casos de Uso

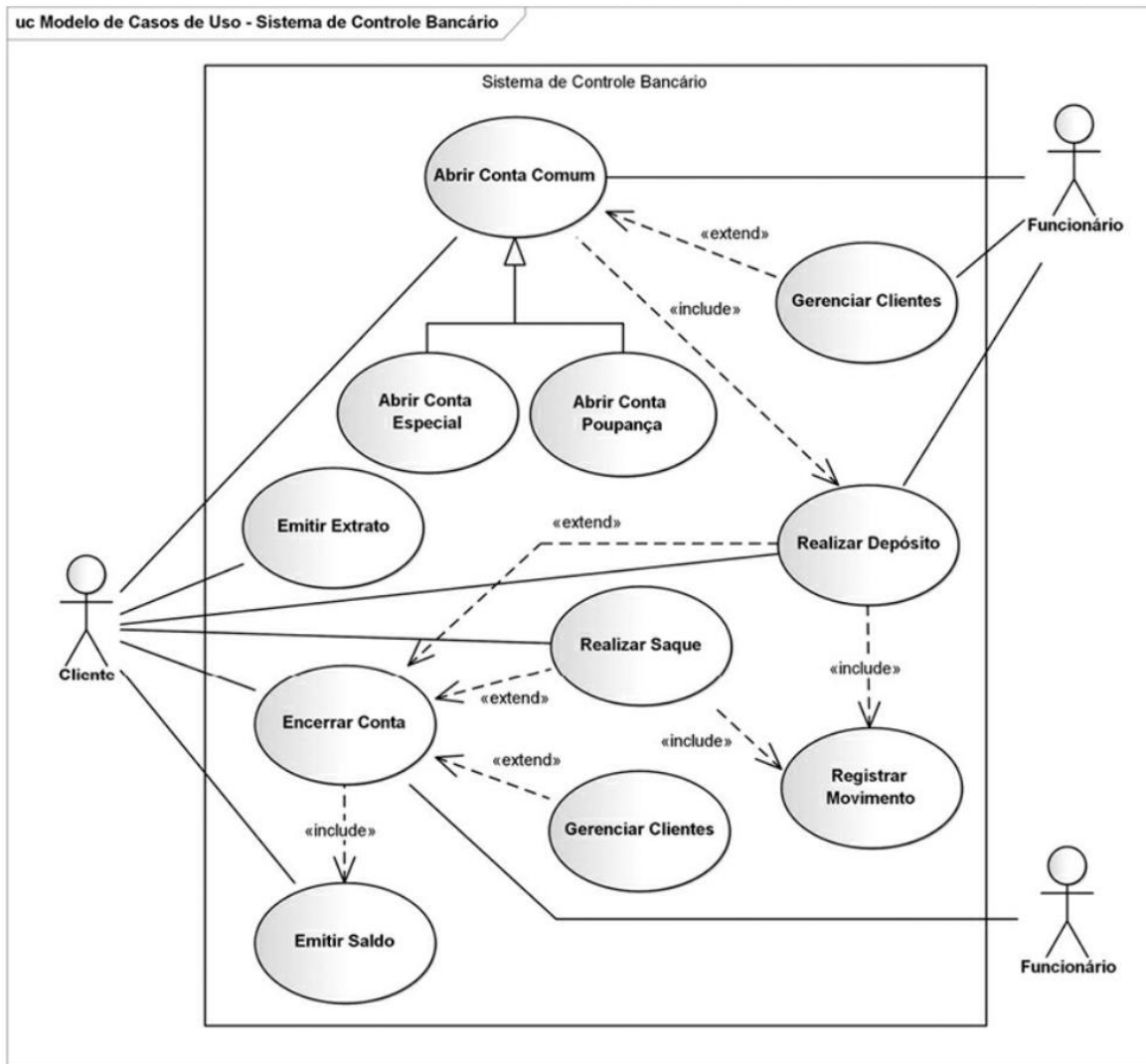
- ❑ O Diagrama de Caso de Uso, **não é o suficiente para representar toda a complexidade das funcionalidades** expressadas pelos casos de uso no diagrama. Para isto, é necessário efetuar a documentação/especificação dos casos de uso que representados neste diagrama.
- ❑ Em adicional, é importante a criação de **Diagramas de Atividade** para representar o fluxo e os atores envolvidos na execução da ação.

Estrutura

- ❑ **Breve descrição:** descreve o papel e o propósito do caso de uso.
- ❑ **Fluxo básico:** descreve o comportamento ideal e primário do sistema.
- ❑ **Fluxos alternativos:** descreve exceções ou desvios do fluxo básico, como a forma que o sistema se comporta quando o ator entra em uma ID de usuário incorreta e a autenticação do usuário falha.
- ❑ **Condições prévias:** um estado do sistema que deve estar presente antes de um caso de uso ser realizado.
- ❑ **Condições posteriores:** uma lista de estados possíveis para o sistema imediatamente após o término do caso de uso.
- ❑ **Pontos de extensão:** um ponto no fluxo de eventos do caso de uso em que outro caso de uso é referenciado.

Item	Descrição do Campo
Nome	Identificador e título do caso de uso. Ex: UC001 - Cadastrar Pedido.
Descrição	Breve explicação sobre o objetivo e o escopo do caso de uso.
Ator Principal	Pessoa ou sistema que inicia o caso de uso.
Atores Secundários	Outros envolvidos que interagem com o sistema durante o caso de uso.
Pré-condições	Condições que devem ser verdadeiras antes do início do caso de uso.
Fluxo Principal	Passos principais seguidos para o sucesso do caso de uso, sem erros.
Fluxo Alternativo	Caminhos alternativos ou variações que podem ocorrer durante a execução do caso.
Fluxo de Exceção	Situações inesperadas ou erros que interrompem o fluxo principal.
Pós-condições	Resultado final após a execução do caso de uso com sucesso.

UML – Diagrama de Caso de Uso



UML – Documentação de Casos de Uso

Documentação do Caso de Uso Abrir Conta Especial

Nome do Caso de Uso		UC02 – Abrir Conta Especial	
Caso de Uso Geral		Abrir Conta Comum	
Ator Principal		Funcionário	
Atores Secundários		Clientes	
Resumo		Este caso de uso descreve as etapas percorridas por um cliente para abrir uma conta-corrente especial com intermediação de um funcionário	
Pré-condições		O pedido de abertura precisa ser aprovado	
Pós-condições		É necessário realizar um depósito inicial	
Cenário Principal			
Ações do Ator		Ações do Sistema	
Idênticas às do caso de uso Abrir Conta Comum, exceto por fornecer comprovante de estar empregado e de que seu salário é superior a R\$ 1.500,00		Idênticas às do caso de uso Abrir Conta Comum, exceto por incluir o limite do cheque especial ao abrir a conta	
Restrições/Validações		1. Para abrir uma conta-corrente especial, é preciso ser maior de idade 2. É necessário comprovar estar empregado e o salário tem que ser superior a R\$ 1.500,00 3. O valor mínimo de depósito inicial é R\$ 50,00	

UML – Documentação de Casos de Uso

Documentação do Caso de Uso Gerenciar Clientes

Nome do Caso de Uso	UC04 – Gerenciar Clientes	
Ator Principal	Funcionário	
Atores Secundários		
Resumo	Este caso de uso descreve as possíveis atividades de manutenção do cadastro de clientes, ou seja, permite incluir, alterar ou consultar clientes. Um cliente não pode ser excluído apenas se tornando inativo	
Pré-condições		
Pós-condições		
	Cenário Principal	
Ações do Ator	Ações do Sistema	
1. Informar o CPF ou CNPJ do cliente	2. Validar CPF/CNPJ	
	3. Consultar o cliente por seu CPF ou CNPJ	
	4. Apresentar dados do cliente	
Restrições/Validações	1. O CPF ou CNPJ precisa ser validado	
	Cenário de Exceção I – CPF/CNPJ inválido	
Ações do Ator	Ações do Sistema	
	1. Informar que o número fornecido não condiz com um CPF/CNPJ válido	
	2. Solicitar um novo número	
	Cenário de Exceção II – Cliente não encontrado	
Ações do Ator	Ações do Sistema	

Continua na próxima página.....

UML – Documentação de Casos de Uso

1. Informar que o CPF/CNPJ fornecido não corresponde a nenhum cliente registrado	
Cenário Alternativo I – Incluir Cliente	
Ações do Ator	Ações do Sistema
1. Informar os dados cadastrais do cliente	
	2. Registrar cliente
Restrições/Validações	1. Os campos nome, endereço e data de nascimento são obrigatórios
Cenário Alternativo II – Alterar Cliente	
Ações do Ator	Ações do Sistema
1. Alterar os dados cadastrais do cliente	
	2. Registrar alterações no cadastro do cliente
Restrições/Validações	1. Os campos nome, endereço e data de nascimento são obrigatórios

UML – Documentação de Casos de Uso

Documentação do Caso de Uso Realizar Depósito

Nome do Caso de Uso		UC05 – Realizar Depósito	
Ator Principal	Funcionário		
Atores Secundários	Clientes		
Resumo	Descreve os passos necessários para um cliente depositar algum valor em uma determinada conta		
Pré-condições			
Pós-condições			
		Cenário Principal	
Ações do Ator		Ações do Sistema	
1. Informar o número da conta			
		2. Verificar se a conta existe	
3. Fornecer o valor a ser depositado			
		4. Somar o valor depositado ao saldo da conta	
		5. Executar caso de uso “Registrar Movimento”	
Restrições/Validações		1. A conta precisa existir e estar ativa	
		Cenário de Exceção – Conta não encontrada	
Ações do Ator		Ações do Sistema	
		1. Comunicar ao cliente que o número da conta informada não foi encontrado	

UML – Documentação de Casos de Uso

Documentação do Caso de Uso Emitir Saldo

Nome do Caso de Uso		UC06 – Emitir Saldo	
Ator Principal	Cliente		
Atores Secundários			
Resumo	Descreve os passos necessários para um cliente obter o saldo referente a uma determinada conta		
Pré-condições			
Pós-condições			
Cenário Principal			
Ações do Ator		Ações do Sistema	
1. Informar o número da conta			
		2. Verificar a existência da conta	
		3. Solicitar a senha da conta	
4. Informar a senha			
		5. Verificar se a senha está correta	
		6. Emitir o saldo	
Restrições/Validações		1. A conta precisa existir e estar ativa	
		2. A senha precisa estar correta	
Cenário de Exceção I – Conta não encontrada			
Ações do Ator		Ações do Sistema	
		1. Comunicar ao cliente que o número da conta informada não foi encontrado	
Cenário de Exceção II – Senha inválida			
Ações do Ator		Ações do Sistema	
		1. Comunicar ao cliente que a senha fornecida não combina com a senha da conta	



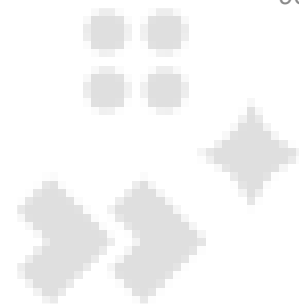
Exercício 03

Efetuar a documentação dos casos de uso “Agendar Consulta” e “Atender Consultas” do exercício 02.

M1 – Atividade em Sala 03



M1 - Atividade em Sala 03



- ❑ Exercícios 01, 02 e 03 deste slide.
- ❑ Data entrega: 19/03 até as 22:30

Próxima Aula

1

Diagrama de Classe

Referências

- ❑ **OBERLEITNER, Allen; MASIERO, Andrey Araujo.** Programação orientada a objetos. São Paulo: i>i, [2021]. E-book. 137 p. (Série Universitária). ISBN 9786555364781.
- ❑ **JANDL JUNIOR, Peter.** Java: guia do programador. 4. ed. São Paulo: Novatec, [2021]. E-book (Kindle). Atualizado para Java 16 (Portuguese Edition).



Dúvidas?

Prof. Maurício Pasetto de Freitas, MSc.
mauriciopasetto@univali.br
(48) 99194 1194

